

Pronóstico de excedencia: hasta dónde alcanza el modelo

MA2003B, equipo 7

Tabla de contenidos

1. Cómo funciona el modelo	1
2. La regla del juego: qué se sabe y cuándo	4
3. Un modelo por horizonte	5
4. El resultado: el pronóstico dura un día	7
5. Dónde está el cuello de botella	8
6. Qué se lleva el reporte	9

`autorregresivo_logistico.pdf` construyó un modelo de *nowcasting*: decide si **hoy** es un día de excedencia usando lo que se midió **hoy**. Alcanza un AUC de 0.928 y su límite está escrito en su propio nombre — no dice nada sobre mañana.

Este documento hace la pregunta siguiente: **¿cuántos días hacia adelante sirve?** No se revisan supuestos, ya se verificaron ahí; se reutiliza la misma especificación y la misma partición, y se cambia sólo el horizonte.

La respuesta corta, para no esconderla: **un día**. Después, el modelo empata con predecir el promedio del mes. Lo interesante es *por qué*, porque la razón no es la que uno esperaría.

1. Cómo funciona el modelo

El proceso es una **cadena de Markov de dos estados**. Cada día está en uno de dos: excedencia ($y_t = 1$) o no ($y_t = 0$). Lo que hace de esto una cadena es que la probabilidad de mañana depende del estado de hoy:

```
library(nanoparquet); library(dplyr); library(pROC); library(ggplot2);
↪ library(tidyr)

af <- read_parquet("../data/processed/af_puntajes_diarios.parquet")
di <- read_parquet("../data/processed/sima_transformado_diario.parquet")
attr(af$fecha, "tzone") <- "UTC"; attr(di$fecha, "tzone") <- "UTC"
af <- rename(af, comb = F1, vent = F2, foto = F3)
```

```
# Misma serie de area metropolitana de autorregresivo_logistico.pdf §0.
m <- inner_join(af, select(di, estacion, fecha, excede_anio_5),
                by = c("estacion", "fecha")) |>
  filter(if_all(c(comb, vent, foto, excede_anio_5), \(v) !is.na(v)))
area <- m |> group_by(fecha) |>
  summarise(comb = mean(comb), vent = mean(vent), foto = mean(foto),
            frac = mean(excede_anio_5), .groups = "drop")
A <- left_join(data.frame(fecha = seq(min(m$fecha), max(m$fecha), by = "day")),
              area, by = "fecha")
interp <- function(v) { i <- which(!is.na(v)); approx(i, v[i], seq_along(v), rule
  ↪ = 2)$y }
for (v in c("comb", "vent", "foto", "frac")) A[[v]] <- interp(A[[v]])
A$y <- as.integer(A$frac >= 0.5)
A$anio <- as.integer(format(A$fecha, "%Y"))

TR <- filter(A, anio <= 2024)
P <- prop.table(table(ayer = head(TR$y, -1), hoy = tail(TR$y, -1)), 1)
round(P, 3)
#>      hoy
#> ayer    0    1
#>    0 0.837 0.163
#>    1 0.379 0.621
```

Se lee por renglones. Si ayer no hubo excedencia, hoy la hay con probabilidad 0.163; si ayer sí la hubo, hoy con 0.621. Esa diferencia —**0.46**— es toda la memoria del proceso: es lo que saber el estado de ayer añade sobre no saberlo.

Qué pasa a dos días, y a tres

Aquí entra la parte útil de la teoría. Para saltar dos días no hace falta un modelo nuevo: se multiplica la matriz por sí misma. La probabilidad de llegar al estado j en dos pasos es la suma sobre todos los caminos posibles —pasar por 0 o pasar por 1— y eso es exactamente lo que hace el producto de matrices:

$$P(y_{t+2} = j \mid y_t = i) = \sum_k P_{ik} P_{kj}, \quad \text{o en forma compacta } \mathbf{P}^2.$$

Esta es la ecuación de Chapman-Kolmogorov, y su consecuencia es que el pronóstico a h días es simplemente $\mathbf{P}^h$.

```
M <- diag(2)
memoria <- do.call(rbind, lapply(1:10, function(h) {
  M <- M %*% P
  data.frame(h = h, desde_0 = M[1, 2], desde_1 = M[2, 2], brecha = M[2, 2] - M[1,
  ↪ 2])
}))
```

```
print(round(memoria, 3), row.names = FALSE)
#>   h desde_0 desde_1 brecha
#> 1  0.163   0.621  0.458
#> 2  0.238   0.448  0.209
#> 3  0.272   0.368  0.096
#> 4  0.288   0.332  0.044
#> 5  0.295   0.315  0.020
#> 6  0.298   0.308  0.009
#> 7  0.300   0.304  0.004
#> 8  0.301   0.303  0.002
#> 9  0.301   0.302  0.001
#> 10 0.301   0.302  0.000
```

La brecha se derrite: 0.458 el primer día, 0.209 el segundo, 0.096 el tercero, y para el sexto ya es 0.009. A partir de ahí **saber lo que pasó hoy no cambia en nada lo que se espera de ese día futuro**: los dos renglones convergen al mismo número, 0.301 —la distribución estacionaria de la cadena, que coincide con la tasa base observada del periodo, 0.300.

Ese desvanecimiento no es casualidad ni un accidente de estos datos. Tiene una fórmula: la brecha decae como $|\lambda_2|^h$, donde λ_2 es el segundo eigenvalor de la matriz.

```
lambda2 <- sort(abs(eigen(P)$values), decreasing = TRUE)[2]
cat(sprintf("segundo eigenvalor lambda2 = %.3f\n\n", lambda2))
#> segundo eigenvalor lambda2 = 0.458
data.frame(h = 1:6, brecha = round(memoria$brecha[1:6], 3),
           lambda2_h = round(lambda2^(1:6), 3)) |> print(row.names = FALSE)
#>   h brecha lambda2_h
#> 1 0.458    0.458
#> 2 0.209    0.209
#> 3 0.096    0.096
#> 4 0.044    0.044
#> 5 0.020    0.020
#> 6 0.009    0.009
```

Las dos columnas son idénticas. Con $\lambda_2 = 0.458$, cada día que pasa recorta la memoria a menos de la mitad. Una cadena así se llama **ergódica**: sin importar de dónde salga, termina olvidando y estabilizándose en su distribución de largo plazo.

```
AZUL <- "#3b6ea5"; ROJO <- "#b5443a"; GRIS <- "#8c8c8c"

# Distribucion estacionaria: el eigenvector izquierdo de lambda = 1. Es a donde
# convergen las dos curvas, y coincide con la media empirica (0.300).
est <- Re(eigen(t(P))$vectors[, 1]); tasa_base <- (est / sum(est))[2]

fig_mem <- memoria |>
  select(h, desde_0, desde_1) |>
  pivot_longer(-h, names_to = "estado", values_to = "p") |>
```

```
mutate(estado = factor(estado, labels = c("hoy sin excedencia",
                                           "hoy con excedencia")))) |>
ggplot(aes(h, p, color = estado)) +
  geom_hline(yintercept = tasa_base, color = GRIS, linetype = "dashed",
            linewidth = .4) +
  geom_line(linewidth = .7) + geom_point(size = 1.6) +
  annotate("text", x = 10, y = tasa_base - .035,
         label = sprintf("estacionaria%.3f", tasa_base),
         hjust = 1, size = 2.6, color = GRIS) +
  scale_color_manual(values = c(AZUL, ROJO), name = NULL) +
  scale_x_continuous(breaks = 1:10) +
  labs(x = "días hacia adelante", y = "P(excedencia)") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "top", panel.grid.minor = element_blank(),
        legend.margin = margin(0, 0, -6, 0))
ggsave("../reports/figuras/pronostico_memoria.pdf", fig_mem, width = 5.5, height =
  ↪ 3)
fig_mem
```

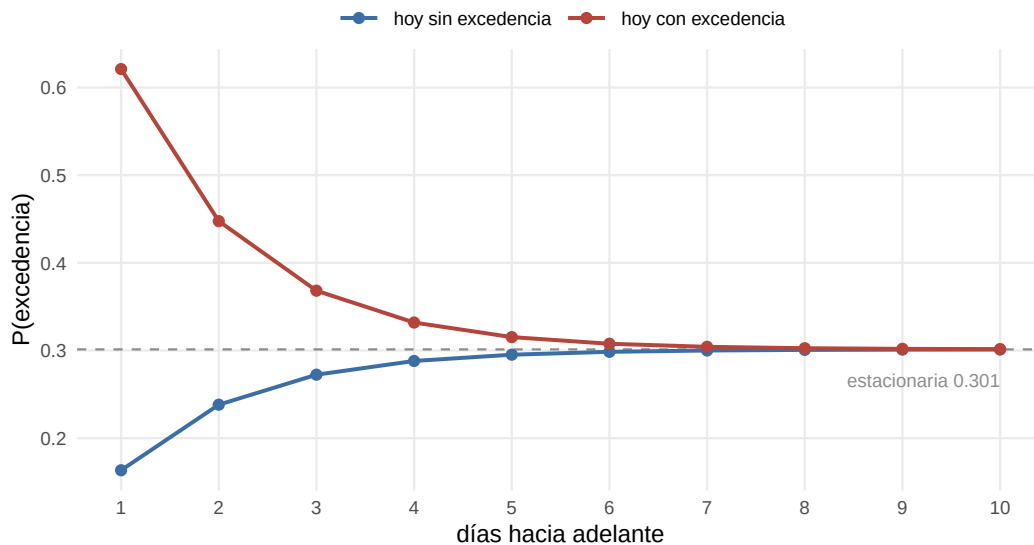


Figura 1: La cadena olvida. Las dos curvas parten de estados opuestos y convergen a la tasa base en unos seis días.

Esto ya pone un techo al pronóstico, y conviene entender de qué tipo. Dice que *la parte autorregresiva* —el rezago de la respuesta— deja de aportar después del tercer día. No dice que el pronóstico entero sea imposible: los factores meteorológicos y el calendario son otra fuente de información, y de ellos se ocupa el resto del documento.

2. La regla del juego: qué se sabe y cuándo

Un pronóstico es honesto sólo si usa información que existía cuando se hizo. La diferencia con el *nowcasting* está justo ahí:

	<i>Nowcasting</i> (día t)	Pronóstico (día $t + h$)
Factores meteorológicos	del día t , medidos	del día t , los últimos conocidos
Excedencia previa	y_{t-1}	y_t
Calendario	del día t	del día $t + h$, se conoce siempre

El calendario es la excepción interesante: la posición del día en el año se sabe con cualquier anticipación, así que los armónicos estacionales entran al pronóstico sin hacer trampa. Los factores meteorológicos no: para predecir el jueves desde el lunes hay que conformarse con la meteorología del lunes.

```
H <- 7
A$y_ult <- lag(A$y, 1)

for (h in 0:H) {
  dia <- as.integer(format(A$fecha + 86400 * h, "%j")) # día del año OBJETIVO
  A[[paste0("s1_", h)]] <- sin(2 * pi * dia / 365.25)
  A[[paste0("c1_", h)]] <- cos(2 * pi * dia / 365.25)
  A[[paste0("s2_", h)]] <- sin(4 * pi * dia / 365.25)
  A[[paste0("c2_", h)]] <- cos(4 * pi * dia / 365.25)
  for (v in c("y", "comb", "vent", "foto"))
    A[[paste0(v, "_f", h)]] <- lead(A[[v]], h) # valor en t+h
}
cat(sprintf("serie:%s dias   ajuste 2021-2024   validacion 2025\n",
            format(nrow(A), big.mark = ",")))
#> serie: 1,642 dias   ajuste 2021-2024   validacion 2025
```

3. Un modelo por horizonte

Hay dos maneras de pronosticar a h días. Una es **iterar** la cadena, como en la sección 1; el problema es que exigiría conocer también los factores futuros, que es justo lo que no se tiene. La otra es **ajustar un modelo directo** por cada horizonte: predecir y_{t+h} con lo disponible en el día t . Es la que se usa aquí, por ser la única que respeta la regla de la sección 2.

Se ajustan cuatro modelos por horizonte para poder comparar:

- **Pronóstico:** calendario del día objetivo + factores del día $t + y_t$.
- **Climatología:** sólo el calendario. Es “predecir el promedio histórico de ese día del año”, y es la línea base que hay que superar para aportar algo.
- **Persistencia:** sólo y_t . Es “mañana será como hoy”.
- **Oráculo:** como el pronóstico, pero con los factores **del día objetivo**. Es imposible en la práctica —requeriría un pronóstico meteorológico perfecto— y sirve como techo: dice cuánto se ganaría si el clima futuro fuera conocido.

```

espec <- function(h, tipo) {
  cal <- paste0(c("s1_", "c1_", "s2_", "c2_"), h)
  rez <- if (h == 0) "y_ult" else "y" # el ultimo y observado en el dia t
  met <- c("comb", "vent", "foto", "I(vent^2)", "I(foto^2)")
  ora <- paste0(c("comb", "vent", "foto"), "_f", h)
  v <- switch(tipo,
    pronostico = c(cal, met, rez),
    climatologia= cal,
    persistencia= rez,
    oraculo = c(cal, ora, rez))
  as.formula(paste("Y ~", paste(v, collapse = " + ")))
}

evaluar <- function(h) {
  D <- A[!is.na(A[[paste0("y_f", h)])] & !is.na(A$y_ult), ]
  D$Y <- D[[paste0("y_f", h)]]
  tr <- D$anio <= 2024; va <- D$anio == 2025
  ajustar <- function(tipo) predict(glm(espec(h, tipo), binomial, D[tr, ]),
    D[va, ], type = "response")
  p <- lapply(c("pronostico", "climatologia", "persistencia", "oraculo"), ajustar)
  names(p) <- c("pron", "clim", "pers", "orac")
  A_ <- \(x) as.numeric(auc(roc(D$Y[va], x, quiet = TRUE)))
  # skill de Brier: 1 = perfecto, 0 = no mejora predecir siempre la tasa base
  brier <- mean((p$pron - D$Y[va])^2)
  nulo <- mean((mean(D$Y[tr]) - D$Y[va])^2)
  data.frame(h = h, AUC = A_(p$pron), climatologia = A_(p$clim),
    persistencia = A_(p$pers), oraculo = A_(p$orac),
    skill = 1 - brier / nulo,
    p_vs_clima = roc.test(roc(D$Y[va], p$pron, quiet = TRUE),
      roc(D$Y[va], p$clim, quiet = TRUE),
      method = "delong")$p.value)
}

tabla <- do.call(rbind, lapply(0:H, evaluar))
print(transform(tabla, AUC = round(AUC, 3), climatologia = round(climatologia, 3),
  persistencia = round(persistencia, 3), oraculo = round(oraculo,
    ↪ 3),
  skill = round(skill, 3), p_vs_clima = round(p_vs_clima, 4)),
  row.names = FALSE)
#> h AUC climatologia persistencia oraculo skill p_vs_clima
#> 0 0.928 0.762 0.737 0.925 0.573 0.0000
#> 1 0.814 0.760 0.741 0.924 0.319 0.0899
#> 2 0.772 0.757 0.661 0.921 0.255 0.5266
#> 3 0.782 0.754 0.659 0.921 0.258 0.0967
#> 4 0.761 0.752 0.635 0.917 0.241 0.4419
#> 5 0.745 0.749 0.633 0.915 0.228 0.5825
#> 6 0.755 0.747 0.643 0.914 0.218 0.3031
#> 7 0.733 0.744 0.641 0.914 0.207 0.1609

```

4. El resultado: el pronóstico dura un día

```
curvas <- tabla |>
  select(h, Oráculo = oraculo, `Pronóstico` = AUC, Climatología = climatologia) |>
  pivot_longer(-h, names_to = "serie", values_to = "auc") |>
  mutate(serie = factor(serie, levels = c("Oráculo", "Pronóstico",
    ↪ "Climatología"),
          labels = c("Con meteorología perfecta", "Pronóstico real",
                     "Climatología"))

fig_hor <- ggplot(curvas, aes(h, auc, color = serie)) +
  geom_line(linewidth = .7) + geom_point(size = 1.6) +
  scale_color_manual(values = c(GRIS, AZUL, ROJO), name = NULL) +
  scale_x_continuous(breaks = 0:7) + ylim(.70, 1) +
  labs(x = "horizonte (días hacia adelante)", y = "AUC en 2025") +
  theme_minimal(base_size = 9) +
  theme(legend.position = "top", panel.grid.minor = element_blank(),
        legend.margin = margin(0, 0, -6, 0))
ggsave("../reports/figuras/pronostico_horizonte.pdf", fig_hor, width = 5.5, height
  ↪ = 3.2)
fig_hor
```

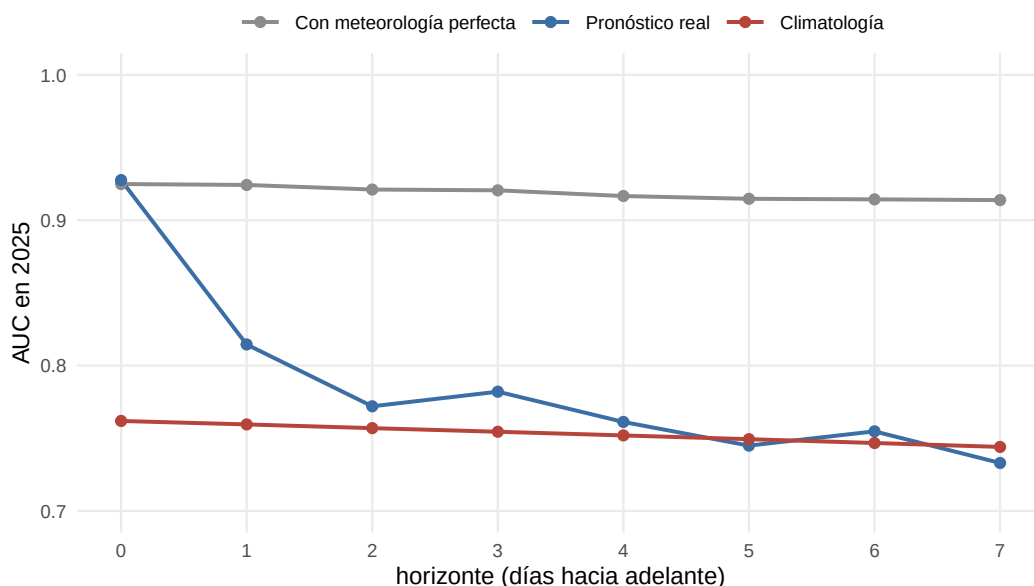


Figura 2: AUC en la validacion de 2025 segun el horizonte. El pronostico real cae hasta confundirse con la climatologia; con meteorologia perfecta se mantendria plano.

La caída es abrupta y luego se aplana:

- **Hoy mismo ($h = 0$), el *nowcasting*:** AUC 0.928. Es la referencia.

- **A un día:** 0.814 contra 0.760. Es la única ventaja de tamaño apreciable que sobrevive, aunque conviene no exagerarla: con 181 días de validación la prueba de DeLong la deja en $p = 0.09$, al filo de lo convencional.
- **A dos y tres días:** 0.772 y 0.782, contra 0.757 y 0.754 de la climatología. Un par de centésimas, indistinguibles del azar.
- **De cuatro días en adelante:** el pronóstico y la climatología son la misma cosa. A siete días el modelo es incluso **peor** (0.733 contra 0.744): arrastra información vieja que ya no sirve y que sólo agrega ruido.

El *skill score* de Brier cuenta la misma historia en otra escala: de 0.573 el mismo día a 0.319 al día siguiente, y de ahí en adelante se estanca cerca de 0.22 — que es esencialmente el mérito de conocer la estación del año, no de tener un modelo.

Conclusión operativa: el modelo pronostica un día. A partir del segundo, la recomendación honesta es usar la climatología, que es más simple y no es peor.

5. Dónde está el cuello de botella

Queda la pregunta de por qué. Hay dos explicaciones posibles y llevan a acciones opuestas:

1. **El proceso es intrínsecamente impredecible** más allá de un día. Si es así, no hay nada que hacer y el modelo ya llegó a su techo.
2. **La meteorología futura es lo que falta.** Si es así, el modelo está bien y el problema es el insumo.

La curva del oráculo separa las dos. Ese modelo es idéntico al pronóstico salvo en un punto: usa los factores del día objetivo, como si alguien pudiera anticipar el clima sin error.

```
data.frame(horizonte = tabla$h,
            pronostico = round(tabla$AUC, 3),
            oraculo = round(tabla$oraculo, 3),
            brecha = round(tabla$oraculo - tabla$AUC, 3)) |> print(row.names =
            ↪ FALSE)
#>   horizonte pronostico oraculo brecha
#>         0      0.928   0.925 -0.003
#>         1      0.814   0.924  0.110
#>         2      0.772   0.921  0.149
#>         3      0.782   0.921  0.139
#>         4      0.761   0.917  0.155
#>         5      0.745   0.915  0.170
#>         6      0.755   0.914  0.160
#>         7      0.733   0.914  0.181
```

El oráculo no decae. A siete días de distancia mantiene un AUC de 0.914, prácticamente el mismo 0.925 que tiene el mismo día. La brecha con el pronóstico real crece de 0 a 0.18 y esa brecha **es** la explicación completa.

La respuesta, entonces, es la segunda: el proceso no es impredecible. Un día de excedencia se explica muy bien por las condiciones meteorológicas de ese día —eso ya lo sabíamos por el *nowcasting*— y

sigue explicándose igual de bien a siete días de distancia. Lo que no se puede hacer es **saber cuáles serán esas condiciones**. El modelo estadístico no es el eslabón débil; el eslabón débil es que la meteorología de pasado mañana no está en estos datos.

De aquí sale la única extensión que valdría la pena: alimentar el modelo con un **pronóstico meteorológico numérico** en lugar de con la última observación. Como los pronósticos de temperatura y viento a 24–72 horas son razonablemente buenos, el resultado real caería en algún punto entre las dos curvas, más cerca del oráculo mientras más corto sea el horizonte. Eso ya no es un problema de métodos multivariados sino de conseguir otra fuente de datos, y queda fuera de este proyecto.

6. Qué se lleva el reporte

- **El pronóstico útil llega a un día:** AUC 0.814 contra 0.760 de la climatología, y aun así con $p = 0.09$ — es la única ventaja apreciable, pero el tamaño de la validación no alcanza para declararla firme. Del segundo día en adelante no hay ventaja, y al séptimo el modelo es peor que el calendario.
- **La memoria de la cadena se agota en tres días.** La brecha entre salir de un día con excedencia y salir de uno limpio cae de 0.458 a 0.096, siguiendo exactamente $|\lambda_2|^h$ con $\lambda_2 = 0.458$.
- **El límite no es del modelo, es del insumo.** Con meteorología perfecta el AUC se mantendría en 0.914 a siete días. Todo el deterioro viene de no conocer el clima futuro.
- **Recomendación:** reportar el *nowcasting* como el producto del proyecto y el pronóstico a un día como extensión. Presentar horizontes más largos sería ofrecer como modelo algo que no supera a un promedio estacional.

Una advertencia que se hereda de `autorregresivo_logistico.pdf` y aplica igual aquí: la validación de 2025 llega sólo hasta el 30 de junio y pertenece a un régimen de excedencia más alto que el promedio del ajuste. Las cifras absolutas de esta tabla son optimistas por la misma razón y en la misma magnitud; las comparaciones **entre** horizontes, que es lo que este documento concluye, no se ven afectadas porque todas comparten la misma validación.